

Variable in C

A **variable** in the C programming language is a named storage location in memory that holds data, which may change during program execution. Variables act as symbolic names (identifiers) for memory locations, enabling programmers to read, write, and manipulate data efficiently.

In essence, variables are the foundation of any programming task in C, as they allow dynamic interaction with data and form the backbone of algorithmic logic.

Definition of Variable

A **variable** in C can be formally defined as:

A variable is an identifier that refers to a memory location where data of a specific type is stored and manipulated.

Each variable in C must be declared with a data type before use, which defines both the **size of memory allocated** and the **range of values** that can be stored.

Characteristics of Variables

1. **Name (Identifier):** A unique symbolic name is assigned to a memory location.
2. **Type (Data Type):** Specifies the nature of data (e.g., integer, floating-point, character).
3. **Value:** The actual data stored in the variable at a given time.
4. **Address:** The physical memory location associated with the variable.

Rules for Naming Variables

When creating variable identifiers, the following rules apply:

- Must begin with a letter or underscore (_).
- Can contain letters, digits, and underscores.
- Cannot use reserved keywords (e.g., int, while, return).
- Case-sensitive (Count and count are different).
- Should be meaningful for readability.

Examples of valid variable names:

age, _value, student_marks, sum2

Examples of invalid variable names:

2value, student-mark, while

Variable Declaration and Initialization

Declaration

A variable must be declared before it can be used.

Declaration informs the compiler of the variable's name and type.

Syntax:

```
data_type variable_name;
```

Example

```
int age; // Declares an integer variable
float salary; // Declares a floating-point variable
```

Initialization

Initialization means assigning an initial value to a variable at the time of declaration.

Syntax:

```
data_type variable_name = value;
```

Example:

```
int age = 25; // Declares and initializes an integer
float pi = 3.1416; // Declares and initializes a float
char grade = 'A'; // Declares and initializes a character
```

Types of Variables in C

The classification of variables in C is crucial because it directly affects **scope**, **lifetime**, **storage location**, and **visibility** within a program. Understanding variable types ensures efficient memory usage, modular program design, and predictable execution behavior.

C broadly classifies variables based on two perspectives:

1. **Scope** – Where the variable can be accessed.
2. **Storage Class** – How long the variable exists in memory and where it is stored.

Local Variables

Definition

- Local variables are declared **inside a function, block, or compound statement** (enclosed within {}) and can be accessed only within that scope.

Characteristics

- Scope: Limited to the block/function in which they are defined.

- Lifetime: Created when the block is entered and destroyed when it is exited.
- Storage Class: **Automatic** (default).
- Initial Value: **Garbage value** if not explicitly initialized.

Example

```
#include <stdio.h>
void display() {
    int x = 10; // Local Variable
    printf("Value of x: %d\n", x);
}
int main() {
    display();
    // printf("%d", x); // Error: x not accessible here
    return 0;
}
```

Use Cases

- Temporary calculations.
- Variables that should not interfere with other functions.
- Modular programming (function-based design).

Global Variables

Definition

Global variables are declared **outside all functions**, usually at the top of the program. They are accessible by **all functions** in the program.

Characteristics

- Scope: Entire program (all functions).
- Lifetime: Program execution duration.
- Storage Class: **Static (by default)**.
- Initial Value: Automatically initialized to 0.

Example

Use Cases

- Sharing data across multiple functions.
- Maintaining program-wide state (e.g., configurations, counters).

```
#include <stdio.h>
int count = 0; // Global Variable

void increment() {
    count++;
}

int main() {
    increment();
    increment();
    printf("Count: %d", count);
    return 0;
}
```

Static Variables

Definition

Static variables are variables declared with the keyword **static**. They retain their value between multiple function calls.

Characteristics

- Scope: Depends on declaration (local static = function scope, global static = file scope).
- Lifetime: Entire program execution.
- Initial Value: Initialized to 0 if not explicitly initialized.
- Storage Class: **Static memory**.

Example

```
#include <stdio.h>
void counter() {
    static int x = 0; // Static Variable
    x++;
    printf("x = %d\n", x);
}

int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

Use Cases

- Maintaining state information within a function.
- Useful in recursive functions.

Avoids reinitialization overhead

Data Types in C Language

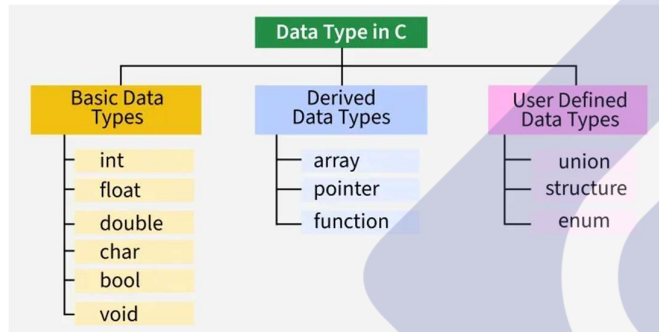
The **C programming language** is often called a **strongly typed language**, meaning that each variable must be declared with a specific **data type** before use.

A **data type** defines:

- The **type of data** (integer, float, character, etc.).
- The **size of memory** allocated.
- The **range of values** that can be stored.
- The **operations** permitted on that variable.

Data types are fundamental because they allow the compiler to interpret how memory should be managed and how values should be manipulated.

Classification of Data Types in C



C data types can be broadly divided into **four categories**:

1. **Basic (Primary) Data Types** – int, char, float, double
2. **Derived Data Types** – arrays, pointers, structures, unions
3. **Enumeration Data Type** – enum (user-defined constants)
4. **Void Data Type** – represents no value

Basic (Primary) Data Types

Integer (int)

- Used to store **whole numbers** (positive, negative, or zero).
- Memory size: **2 bytes** (16-bit compiler) / **4 bytes** (32-bit & 64-bit compilers).
- Range (for 4 bytes): **-2,147,483,648 to +2,147,483,647**.

Example:

```
int age = 25;
printf("Age = %d", age);
```

Character (char)

- Used to store a **single character** (letters, digits, symbols).
- Memory size: **1 byte**.
- Range: **-128 to 127** (signed char), **0 to 255** (unsigned char).

Example:

```
char grade = 'A';
printf("Grade = %c", grade);
```

Floating Point (float)

- Used to store **decimal numbers** with single precision (6 digits of accuracy).
- Memory size: **4 bytes**.
- Range: **3.4E-38 to 3.4E+38**.

Example:

```
float pi = 3.14159;
printf("Value of Pi = %f", pi);
```

Double Precision (double)

- Used to store **decimal numbers** with double precision (15 digits of accuracy).
- Memory size: **8 bytes**.
- Range: **1.7E-308 to 1.7E+308**.

Example:

```
double distance = 9876543.123456;
printf("Distance = %lf", distance);
```

Modified Data Types

C allows modifying basic data types with **type qualifiers** like short, long, signed, and unsigned.

Data Type	Size (bytes)	Range (on 32-bit system)
short int	2	-32,768 to 32,767
unsigned short	2	0 to 65,535
int	4	-2,147,483,648 to 2,147,483,647
unsigned int	4	0 to 4,294,967,295
long int	4 or 8	-2,147,483,648 to 2,147,483,647 (or larger)

Data Type	Size (bytes)	Range (on 32-bit system)
unsigned long	4 or 8	0 to 4,294,967,295 (or larger)
long long	8	−9,223,372,036,854,775,808 to 9,223,372,036...
float	4	3.4E−38 to 3.4E+38
double	8	1.7E−308 to 1.7E+308
long double	10–16	Higher precision than double

Derived Data Types

Derived from primary data types:

- **Arrays:** Collection of elements of the same data type.

```
int marks[5] = {90, 85, 88, 92, 76};
```

Pointers: Variables that store memory addresses.

```
int x = 10;
int *ptr = &x;
```

Structures: Collection of variables of different types under a single name.

```
struct student {
    int id;
    char name[20];
};
```

Structures: Collection of variables of different types under a single name.

```
struct student {
    int id;
    char name[20];
};
```

Unions: Similar to structures, but share the same memory location.

```
union data {
    int i;
    float f;
};
```

Constants in C Language

In C programming, a **constant** refers to a **fixed value** that does not change during the execution of a program. Unlike variables, whose values can be modified, constants are **immutable identifiers** representing specific data.

Constants are essential for:

- Improving **readability** of code.
- Ensuring **data integrity**.
- Avoiding **accidental modifications**.
- Making programs more **maintainable and reliable**.

Definition

A constant in C is a data item whose value is **fixed at compile time and cannot be altered during program execution**.

Constants are stored in memory just like variables, but their values remain **unchanged** throughout the program.

Types of Constants in C

C provides several categories of constants:

1. **Integer Constants**
2. **Real (Floating-Point) Constants**
3. **Character Constants**
4. **String Constants**
5. **Enumeration Constants**
6. **Symbolic Constants** (defined using **#define** or **const** keyword)

Integer Constants

- Represent whole numbers without fractional parts.
- Can be written in **decimal, octal, or hexadecimal** form.

Examples

```
int a = 10; // Decimal constant
int b = 015; // Octal constant (prefix 0 → 13 in decimal)
int c = 0x1A; // Hexadecimal constant (prefix 0x → 26 in decimal)
```

Real (Floating-Point) Constants

- Represent numbers with fractional parts or exponential notation.

Examples

```
float pi = 3.14159; // Fractional form
double exp = 1.2e3; // Exponential form (1.2 × 10³ = 1200.0)
```

Character Constants

- Represent a **single character** enclosed within single quotes (' ').
- Stored as an **integer ASCII value** internally

Examples

```
char grade = 'A'; // Character constant
char newline = '\n'; // Escape sequence constant
```

String Constants

- Represent a **sequence of characters** enclosed in double quotes (" ").
- Stored as a **null-terminated character array** in memory.

Examples

```
char name[] = "C Language";
printf("%s", name);
```

Symbolic Constants

Using #define Preprocessor Directive

- Creates symbolic names for constants.
- Value cannot be changed throughout the program.

Example:

```
#include <stdio.h>
#define PI 3.14159
int main() {
    printf("Value of PI = %f", PI);
    return 0;
}
```

Using const Keyword

- Declares a variable as constant.
- Compiler prevents modification.

Example:

```
#include <stdio.h>
int main() {
    const int MAX = 100;
    printf("MAX = %d", MAX);
    // MAX = 200; // Error: assignment of read-only
    variable
    return 0;
}
```

Escape Sequence Constants

Special character constants used for formatting output.

Escape Sequence Meaning

\n	New line
\t	Horizontal tab
\\	Backslash
\"	Double quote
\'	Single quote
\0	Null character

Literals in C Language

In C programming, the terms **constants** and **literals** are often used interchangeably, but there is a subtle difference:

- **Constant** → A named entity whose value cannot change during program execution (e.g., `const int x = 10;`).
- **Literal** → The **actual fixed value** assigned to a constant or variable in the code (e.g., 10 in `int x = 10;`).

Thus, **literals are the raw values written directly in the program source code**, while constants are the identifiers (names) representing those fixed values.

```
int a = 25; // Here 25 is an integer literal
char grade = 'A'; // 'A' is a character literal
float pi = 3.14; // 3.14 is a floating-point literal
```


Types of Literals in C

C supports several kinds of literals:

1. **Integer Literals**
2. **Floating-Point Literals**
3. **Character Literals**
4. **String Literals**

Integer Literals

- Represent whole numbers (positive or negative).
- Can be written in **decimal**, **octal**, or **hexadecimal** form.
- May also include a **suffix** to specify type (U for unsigned, L for long).

```
int a = 100;    // Decimal literal
int b = 012;    // Octal literal (prefix 0) → 10 in decimal
int c = 0x1F;   // Hexadecimal literal (prefix 0x) → 31 in decimal
unsigned int d = 50U; // Unsigned literal
long int e = 1000L; // Long literal
```

Floating-Point Literals

- Represent real numbers with decimal points or exponential notation.
- May have suffix f (float), l (long double).

Examples

```
float pi = 3.14159f; // Float literal
double exp = 1.2e3;  // Exponential notation (1.2 × 10³ = 1200.0)
long double big = 3.14L; // Long double literal
```

Character Literals

- Represent a single character enclosed within single quotes (' ').
- Stored internally as an **integer ASCII value**.

Examples

```
char grade = 'A'; // Character literal
char digit = '9'; // Character literal
char newline = '\n'; // Escape sequence literal
```

String Literals

- Represent a sequence of characters enclosed in double quotes (" ").
- Stored as an array of characters with a **null character** \0 at the end.

Examples

```
char name[] = "C Language"; // String literal
printf("%s", "Hello World"); // Direct use of string literal
```

Difference Between Constants and Literals in C

Feature	Constants	Literals
Definition	A named identifier that refers to a fixed value and cannot be modified.	A fixed raw value written directly in the program code.
Nature	Symbolic representation of a value.	The actual value itself.
Declaration	Declared using const keyword or #define preprocessor directive.	Written directly in code without declaration.
Storage	Stored in memory with an identifier (name).	Embedded directly in the compiled code (literal constants in data section).
Modifiability	Value cannot be modified after declaration.	Value itself is fixed, cannot be altered.
Reusability	Can be reused multiple times by referring to its name.	Each occurrence must be written explicitly wherever needed.
Examples in C	const int MAX = 100;	
#define PI 3.14	100, 3.14, 'A', "Hello"	
Readability	Improves readability by giving meaningful names to values.	Less readable if used repeatedly without context.
Maintainability	Easy to update (change once, reflects everywhere).	Hard to update (every occurrence must be changed manually).
Use Case	When a value is used multiple times or must be protected from modification.	When a single raw value is required directly in expressions.

Comparison of Local, Global, and Static Variables in C

Feature	Local Variable	Global Variable	Static Variable
Definition	Declared inside a function or block, accessible only within it.	Declared outside all functions, accessible by all functions in the program.	Declared with static keyword; retains its value between function calls.
Scope	Limited to the block or function where it is declared.	Entire program (all functions can access it).	If inside a function → scope limited to that function. If outside → scope limited to that file.
Lifetime	Exists only during the execution of the function/block.	Exists throughout the entire program execution.	Exists for the entire program execution (but scope depends on declaration location).
Storage Location	Stored in stack memory.	Stored in data segment (static memory).	Stored in data segment (static memory).
Default Value	Garbage (undefined) if not initialized.	Automatically initialized to 0 if not explicitly initialized.	Automatically initialized to 0 if not explicitly initialized.
Keyword Used	auto (optional, since auto is default).	None required (declared normally).	static keyword.
Initialization	Must be done manually by the programmer.	Can be initialized once; otherwise set to 0 automatically.	Initialized only once; value is retained across multiple function calls.
Re-initialization	Re-initialized every time the function/block is executed.	Can be re-initialized anytime in program execution.	Not re-initialized on function calls; maintains last stored value.
Use Cases	Temporary/short-term variables for computations within a function.	Shared values across multiple functions (e.g., configuration, counters).	Maintaining state information across multiple function calls (e.g., recursive calls, counters).